

ASSESSMENT TOOL 3:

Mock Interview

5. Video Stream Drops Frames During Processing

Answer:

When a video stream drops frames during processing, it means that the system cannot process and display frames at the same speed at which they are being captured. This usually happens when the processing workload is too high compared with the available CPU, GPU, or memory resources.

Possible Causes:

- High-resolution video frames require more processing time.
- Complex image-processing or object-detection algorithms may be computationally expensive.
- The system may have limited CPU or GPU resources.
- Frames may be captured and processed sequentially, creating delays.
- Inefficient memory usage or unnecessary image conversions can increase latency.

Redesigned Approach:

A better pipeline separates the major operations into different stages:

Video Capture → Frame Buffer → Frame Processing → Visualization → Display

The video-capture stage continuously receives frames, while the processing stage handles the required computer-vision operations. A buffer or queue can temporarily store frames between these stages. Multithreading can also be used so that capturing and processing do not unnecessarily block each other.

Optimization Techniques:

1. Reduce the resolution of the video frames.
2. Skip unnecessary frames when complete frame-by-frame processing is not required.
3. Use efficient OpenCV operations instead of repeated expensive calculations.
4. Use GPU acceleration when available.
5. Process only the required region of interest instead of the complete frame.
6. Use buffering or multithreading to separate capture and processing.

Trade-Off:

These optimizations improve real-time performance, but reducing resolution or skipping frames can decrease visual quality or detection accuracy. Therefore, the system should choose a balance between **processing speed, accuracy, and video quality**.

13. Simple Face Recognition Demo

Answer:

A simple face recognition system can be designed as a sequence of stages that first detects a face and then identifies the person. The basic pipeline is:

Image/Video Input → Face Detection → Face Preprocessing → Face Recognition → Result Visualization

1. Input Acquisition:

The system receives an image or live video from a webcam. OpenCV can be used to capture the input and divide video into individual frames.

2. Face Detection:

The system searches each frame for human faces. A face detector such as Haar Cascade or another lightweight detector can identify the location of each face.

3. Face Preprocessing:

After detecting a face, the corresponding region is cropped from the frame. The face can then be resized and converted to an appropriate format, such as grayscale, depending on the recognition method.

4. Face Recognition:

The processed face is compared with previously trained or registered faces. A suitable recognition algorithm produces the predicted identity and a confidence or similarity measure.

5. Visualization:

The recognized person's name can be displayed along with a bounding box around the face. If the face does not sufficiently match a known identity, it can be classified as unknown.

Design Justification:

Separating face detection from face recognition makes the system easier to understand and maintain. A lightweight detector and recognition method are suitable for a simple demonstration because they require fewer computational resources and can operate effectively on a small dataset.

4. Image Appears Too Dark When Read

Answer:

An image may appear too dark because of poor illumination, incorrect camera exposure, low pixel intensity, or an unsuitable image-processing pipeline. A dark image can make important objects and features difficult to identify.

The improved processing pipeline can be represented as:

Image Input → Image Validation → Brightness Adjustment → Contrast Enhancement → Histogram Enhancement → Output

1. Image Validation:

First, the system should verify that the image has been loaded correctly. An incorrectly loaded image should not be processed further.

2. Brightness Adjustment:

The brightness of the image can be increased by modifying the pixel intensity values. This makes dark regions more visible.

3. Contrast Enhancement:

Contrast enhancement increases the difference between darker and brighter regions. This helps reveal important structures that may not be clearly visible in the original image.

4. Histogram Equalization:

For grayscale images, histogram equalization can redistribute intensity values across a wider range. This can improve overall visibility and contrast.

5. CLAHE:

For images containing uneven lighting, Contrast Limited Adaptive Histogram Equalization (CLAHE) can improve local contrast. It works on smaller regions of the image and limits excessive contrast enhancement.

Advantages:

The modified pipeline improves image visibility, reveals hidden details, and produces a better input for later operations such as edge detection, feature detection, and object recognition.

The enhancement should be controlled carefully because excessive brightness or contrast can remove useful information or create unwanted artifacts.

10. Drawing Bounding Boxes and Labels for Detected Objects

Answer:

A bounding-box and labeling module is used to visually represent objects detected by a computer-vision system. The detector normally provides the **object location, object class, and confidence score**.

The overall pipeline is:

Input Frame → Object Detection → Bounding-Box Coordinates → Draw Box → Add Label → Display/Save Output

1. Object Detection:

The system processes the input image or video frame using an object-detection algorithm. The detector identifies objects and returns their locations.

2. Obtain Detection Information:

For each detected object, the system obtains:

- Bounding-box coordinates
- Object class
- Confidence score

3. Draw Bounding Box:

A rectangular box is drawn around the detected object using the coordinates provided by the detector. This allows the user to visually identify the object's location.

4. Add Labels:

A label containing the object name can be placed near the bounding box. The confidence score may also be displayed to indicate how confident the detector is about its prediction.

Design Justification:

The detection and visualization components should be designed as separate modules. This allows the same visualization module to work with different object-detection algorithms. It also improves code organization, maintainability, and scalability.

This approach is useful in applications such as **traffic monitoring, surveillance, industrial inspection, and object tracking**.

7. Processing Both Images and Videos Using the Same Pipeline

Answer:

A generalized OpenCV system can process both images and videos by designing the processing stage around a common concept called a **frame**. An image is treated as a single frame, while a video is treated as a sequence of frames.

The generalized pipeline is:

Input → Frame Generation → Preprocessing → Detection/Processing → Visualization → Output

1. Input Handling:

The system first determines whether the input is an image or video.

For images, the image is loaded once. For videos, frames are continuously captured from a video file or camera.

2. Frame Generation:

Both types of input are converted into a common frame format. This allows the same processing functions to be applied to both.

3. Preprocessing:

Operations such as resizing, color conversion, noise removal, and enhancement can be applied uniformly to each frame.

4. Detection and Processing:

The same computer-vision algorithms can then be applied to the frame. For example, the system can perform edge detection, object detection, feature extraction, or face detection.

5. Output:

For an image, the final result is produced once. For a video, the processed frames are displayed continuously or saved as a new video.

Design Justification:

This generalized architecture reduces code duplication because the core processing functions are shared by both images and videos. It improves **modularity, reusability, maintainability, and scalability**.

The main difference is that an image requires only one processing cycle, whereas a video repeatedly sends each frame through the same pipeline.

